GigGuard

Relational Data Model & ER Diagram

AI-Powered Parametric Income Insurance · India Gig Economy

---

1. Overview

GigGuard's data model consists of 5 core entities. Every piece of data in the system traces back to a Worker — the central entity. The model is designed for two priorities: instant claim processing (automated pipeline from trigger → claim → payout) and fraud detection (denormalized fields to enable fast anomaly queries without expensive joins).

Entity	Color code	Purpose
WORKERS	`Blue (PK source)`	Central entity. Every other record links back to a worker.
POLICIES	Teal (weekly)	One record per worker per week. Stores premium breakdown and coverage amount.
CLAIMS	Amber (triggered)	Auto-created when a parametric trigger fires. One per worker per event.
PAYOUTS	Purple (financial)	One payout per approved claim. Stores Razorpay transaction details.
DISRUPTION_EVENTS	Coral (aggregate)	Logs each trigger breach. One event → many claims across all affected workers.

Design principle: denormalization is intentional

→ worker_id is stored on both CLAIMS and PAYOUTS (not just via POLICIES) so a dashboard query for "all claims by worker X" never needs a multi-table join.

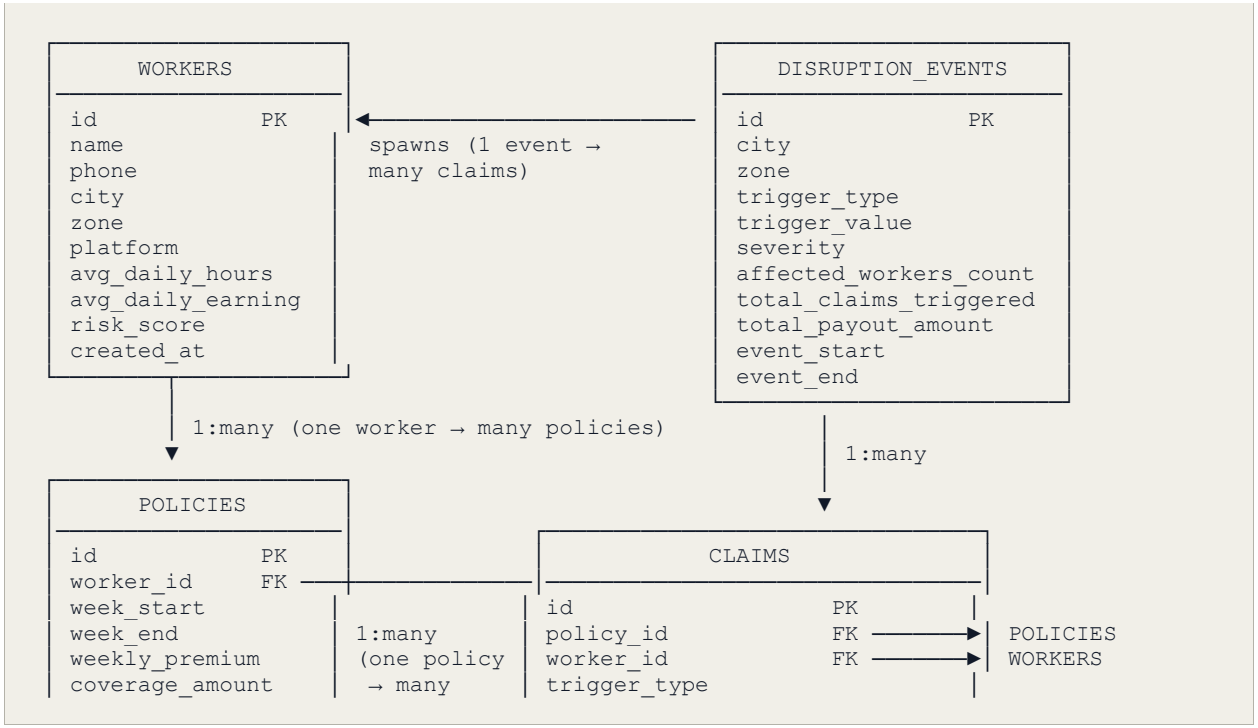
→ fraud_reasons is JSONB (not a child table) because reasons are always read together with the claim and never queried individually.

→ DISRUPTION_EVENTS stores aggregate totals (affected_workers_count, total_payout_amount) to power the insurer dashboard without summing across thousands of claims at query time.

---

2. Entity-Relationship Diagram

Crow's foot notation. PK = primary key, FK = foreign key.



```
│ status              │  claims)     │ trigger_value                  │
│ base_rate           │              │ trigger_threshold              │
│ zone_multiplier     │              │ disruption_hours               │
│ weather_multiplier  │              │ payout_amount                  │
│ history_multiplier  │              │ fraud_score                    │
│ created_at          │              │ is_flagged                     │
└─────────────────────┘              │ fraud_reasons  (JSONB)         │
                                     │ status                         │
                                     │ created_at                     │
                                     │ paid_at                        │
                                     └────────────────────────────────┘
                                                  │
                                                  │ 1:1 (one claim → one payout)
                                                  ▼
                                     ┌──────────────────────────────┐
                                     │           PAYOUTS            │
                                     ├──────────────────────────────┤
                                     │ id                  PK       │
                                     │ claim_id            FK       │
                                     │ worker_id           FK       │
                                     │ amount                       │
                                     │ payment_method               │
                                     │ razorpay_payout_id           │
                                     │ status                       │
                                     │ created_at                   │
                                     │ completed_at                 │
                                     └──────────────────────────────┘
```

---

3. Entity Definitions

3.1 WORKERS

The central entity. Every policy, claim, and payout in the system has a direct or indirect FK to a worker. The risk_score (0.0–1.0) is computed by the ML premium model on registration and updated weekly.

WORKERS (table)			
Column name	**Data type**	**Constraint**	**Description**
id	UUID	**PK**	Auto-generated primary key (gen_random_uuid())
name	VARCHAR(100)	NOT NULL	Full name of the delivery partner
phone	VARCHAR(15)	UNIQUE NN	Mobile number — used as login identifier
city	VARCHAR(50)	NOT NULL	One of: Mumbai, Delhi, Chennai, Bangalore, Hyderabad
zone	VARCHAR(100)	NOT NULL	Hyper-local delivery zone within the city
platform	VARCHAR(20)	NOT NULL	zomato or swiggy
avg_daily_hours	DECIMAL(4,2)	NOT NULL	Average hours worked per day (used in payout formula)
avg_daily_earning	DECIMAL(10,2)	NOT NULL	Average daily income in ₹ (used in coverage calculation)
risk_score	DECIMAL(5,4)	DEFAULT 0.5	ML-computed 0.0–1.0 risk score. Updated on each policy purchase.
created_at	TIMESTAMPTZ	DEFAULT NOW	Registration timestamp

3.2 POLICIES

One record per worker per calendar week. Stores the complete premium breakdown so judges can see exactly how the AI model computed the final price. A worker cannot have two active policies for the same week (enforced by application logic).

POLICIES (table)			
Column name	**Data type**	**Constraint**	**Description**
id	UUID	**PK**	Auto-generated primary key
worker_id	UUID	**FK → WORKERS**	Which worker this policy belongs to

week_start	DATE	NOT NULL	Monday of the coverage week
week_end	DATE	NOT NULL	Sunday of the coverage week
weekly_premium	DECIMAL(10,2)	NOT NULL	Final premium in ₹ paid by the worker
coverage_amount	DECIMAL(10,2)	NOT NULL	Maximum payout this week (capped at ₹800)
status	VARCHAR(20)	DEFAULT active	active | expired | claimed
base_rate	DECIMAL(10,2)	DEFAULT 35	Fixed base rate (₹35). Stored for audit trail.
zone_multiplier	DECIMAL(5,4)	NOT NULL	ML-computed zone risk factor (0.80–1.40)
weather_multiplier	DECIMAL(5,4)	NOT NULL	7-day forecast factor (0.90–1.30)
history_multiplier	DECIMAL(5,4)	NOT NULL	Claim history factor (0.85–1.25)
created_at	TIMESTAMPTZ	DEFAULT NOW	Policy activation timestamp

3.3 CLAIMS

Auto-created by the trigger monitor when a parametric threshold is breached. Contains both raw trigger data (for transparency) and ML fraud analysis. The status field tracks the automated pipeline from detection to payout.

CLAIMS (table)

Column name	Data type	Constraint	Description
id	UUID	**PK**	Auto-generated primary key
policy_id	UUID	**FK → POLICIES**	Which policy this claim is against
worker_id	UUID	**FK → WORKERS**	Denormalized FK for fast worker-level queries
trigger_type	VARCHAR(50)	NOT NULL	heavy_rain | severe_aqi | extreme_heat | flood_alert | curfew_strike
trigger_value	DECIMAL(10,4)	NOT NULL	Actual measured value (e.g. 18.2 mm/hr)

trigger_threshold	DECIMAL(10,4)	NOT NULL	Threshold that was breached (e.g. 15.0 mm/hr)
disruption_hours	DECIMAL(4,2)	NOT NULL	Estimated hours worker couldn't work (from formula)
payout_amount	DECIMAL(10,2)	NOT NULL	(avg_daily_earning/8) × disruption_hours × 0.8
fraud_score	DECIMAL(5,4)	NULLABLE	ML anomaly score: 0.0 = clean, 1.0 = very suspicious
is_flagged	BOOLEAN	DEFAULT false	True if fraud_score > 0.65 → sent to manual review
fraud_reasons	JSONB	DEFAULT []	Array of reason strings from fraud scorer
status	VARCHAR(20)	DEFAULT triggered	triggered → validating → approved → paid | rejected
created_at	TIMESTAMPTZ	DEFAULT NOW	When the trigger fired and claim was created
paid_at	TIMESTAMPTZ	NULLABLE	When the Razorpay payout completed

3.4 PAYOUTS

One payout record per approved claim (1:1 relationship). Stores the Razorpay transaction reference and tracks payment status independently of the claim status — allowing payment retries without duplicating claim records.

PAYOUTS (table)

Column name	Data type	Constraint	Description
id	UUID	**PK**	Auto-generated primary key
claim_id	UUID	**FK → CLAIMS**	The claim that triggered this payout
worker_id	UUID	**FK → WORKERS**	Denormalized FK for fast worker payment history queries
amount	DECIMAL(10,2)	NOT NULL	Payout amount in ₹ (same as claim.payout_amount)

payment_method	VARCHAR(20)	DEFAULT upi	upi | bank_transfer
razorpay_payout_id	VARCHAR(100)	NULLABLE	Razorpay transaction ID (null for mock/sandbox)
status	VARCHAR(20)	DEFAULT pending	pending | processing | paid | failed
created_at	TIMESTAMPTZ	DEFAULT NOW	When payout was initiated
completed_at	TIMESTAMPTZ	NULLABLE	When Razorpay confirmed the transfer

3.5 DISRUPTION_EVENTS

Logs each external trigger breach at the city/zone level. This is the event source that spawns claim records. Stores running aggregate totals so the insurer dashboard can show loss ratios without summing across thousands of claim rows at query time.

DISRUPTION_EVENTS (table)

Column name	Data type	Constraint	Description
id	UUID	**PK**	Auto-generated primary key
city	VARCHAR(50)	NOT NULL	Affected city name
zone	VARCHAR(100)	NULLABLE	Hyper-local zone (null = entire city affected)
trigger_type	VARCHAR(50)	NOT NULL	Same enum as claims.trigger_type
trigger_value	DECIMAL(10,4)	NOT NULL	The measured value that breached the threshold
severity	VARCHAR(20)	NOT NULL	moderate | severe | extreme (computed from value)
affected_workers_count	INTEGER	DEFAULT 0	How many workers had active policies at breach time
total_claims_triggered	INTEGER	DEFAULT 0	Incremented as each worker's claim is created

total_payout_amount	DECIMAL(12,2)	DEFAULT 0	Running total of all payouts from this event
event_start	TIMESTAMPTZ	NOT NULL	When the trigger threshold was first breached
event_end	TIMESTAMPTZ	NULLABLE	When conditions returned below threshold (null = ongoing)

---

4. Relationships

From entity	Cardinality	To entity	Type	Business meaning
WORKERS	1 : many	POLICIES	identifying	One worker buys one new policy each week. Historical policies are retained.
WORKERS	1 : many	CLAIMS	non-identifying	worker_id denormalized on claims for fast lookup without joining POLICIES.
WORKERS	1 : many	PAYOUTS	non-identifying	worker_id denormalized on payouts for fast payment history queries.
POLICIES	1 : many	CLAIMS	identifying	One policy can have multiple claims (different trigger types in the same week).
CLAIMS	1 : 1	PAYOUTS	identifying	Every approved claim produces exactly one payout. Enforced by application logic.
DISRUPTION_EVENTS	1 : many	CLAIMS	non-identifying	One breach event spawns one claim per affected worker in the zone.

Cardinality rules

→ A worker can have 0 or 1 active policy per week (application enforces uniqueness on worker_id + week_start).

→ A policy can have 0 to 5 claims (one per trigger type — a worker cannot claim the same trigger twice in one week).

→ A claim can have 0 or 1 payouts (0 if still pending/rejected, 1 once approved).

→ A disruption event can have 0 to N claims where N = number of workers with active policies in the affected zone.

---

5. Indexes

All indexes are non-unique unless noted. Chosen to support the three most frequent query patterns: worker dashboard load, trigger monitor deduplication check, and insurer analytics aggregation.

Index name	Table (column)	Query pattern	Why needed
idx_policies_worker_id	policies(worker_id)	Dashboard load	Get all policies for a worker instantly
idx_policies_status	policies(status)	Monitor query	Find all active policies when trigger fires
idx_claims_worker_id	claims(worker_id)	Dashboard load	Get claim history for a worker without joining POLICIES
idx_claims_status	claims(status)	Monitor query	Find pending claims needing fraud scoring
idx_claims_created_at	claims(created_at)	Analytics	Time-range queries for insurer loss ratio reports
idx_disruption_city	disruption_events(city)	Deduplication	Check for recent same-city events before creating new one

---

6. Status Field Transitions

policies.status

From	→	To	Trigger
(new)	→	active	Worker activates weekly cover by paying the premium
active	→	expired	Cron job runs at week end — no claims were filed
active	→	claimed	At least one claim for this policy reaches status = paid

claims.status

From	→	To	Trigger
(new)	→	triggered	Trigger monitor detects threshold breach and creates the record
triggered	→	validating	ML fraud scorer returns fraud_score > 0.65 → manual review queue
triggered	→	approved	ML fraud scorer returns fraud_score ≤ 0.65 → auto-approved
validating	→	approved	Human reviewer clears the claim in insurer dashboard
validating	→	rejected	Human reviewer rejects the claim
approved	→	paid	Razorpay payout API call succeeds → paid_at timestamp set

payouts.status

From	→	To	Trigger
(new)	→	pending	Payout record created when claim is approved

From	→	To	Trigger
pending	→	processing	Razorpay API call initiated
processing	→	paid	Razorpay webhook confirms transfer success
processing	→	failed	Razorpay webhook reports failure → retried once automatically

---

7. Sample Data

Representative rows showing how the tables look after a simulated rainstorm in Delhi.

workers (1 row — Ravi Kumar)

id	name	city	zone	platform	avg_daily_earning	risk_score	created_at
w-001...	Ravi Kumar	Delhi	Lajpat Nagar	zomato	₹720.00	0.7200	2026-03-10

policies (Ravi's active policy this week)

id	worker_id	weekly_premium	coverage_amount	status	zone_mult	weather_mult
p-002...	w-001...	₹44.00	₹4,032.00	active	1.2000	1.1000

claims (auto-created after Tuesday rainstorm)

id	policy_id	trigger_type	trigger_value	payout_amount	fraud_score	status
c-003...	p-002...	heavy_rain	18.2 mm/hr	₹180.00	0.0800	paid

payouts (Razorpay transfer for the above claim)

id	claim_id	amount	payment_method	razorpay_payout_id	status
py-004...	c-003...	₹180.00	upi	rzp_pay_mock_001	paid

---

8. Key SQL Queries

Queries that power the two dashboards. All use the denormalized FKs to avoid joins.

Worker dashboard — load everything in one query

```
SELECT w.name, w.city, w.zone, w.risk_score,
       p.weekly_premium, p.coverage_amount, p.status AS policy_status,
       p.week_start, p.week_end,
       c.id AS claim_id, c.trigger_type, c.payout_amount, c.status AS
claim_status,
       py.razorpay_payout_id, py.completed_at
FROM   workers w
LEFT   JOIN policies p ON p.worker_id = w.id AND p.status = 'active'
LEFT   JOIN claims   c ON c.worker_id = w.id
LEFT   JOIN payouts  py ON py.claim_id = c.id
WHERE  w.id = $1
ORDER  BY c.created_at DESC;
```

Insurer dashboard — loss ratio this month

```
SELECT
  SUM(p.weekly_premium)       AS total_premiums,
  SUM(py.amount)              AS total_payouts,
  ROUND(SUM(py.amount) /
        NULLIF(SUM(p.weekly_premium),0) * 100, 2) AS loss_ratio_pct
```

```
FROM   policies p
LEFT   JOIN claims  c  ON c.policy_id = p.id AND c.status = 'paid'
LEFT   JOIN payouts py ON py.claim_id = c.id
WHERE  p.week_start >= DATE_TRUNC('month', NOW());
```

Trigger monitor — deduplication check

```
SELECT id FROM disruption_events
WHERE  city         = $1
AND    trigger_type = $2
AND    event_start  > NOW() - INTERVAL '6 hours'
AND    event_end    IS NULL
LIMIT  1;

-- Returns a row → same event still active → skip creating new claims
-- Returns no rows → new event → create disruption_event + claims
```